Fast Recovery in Software-Defined Networks

Vinaybaalaji PNS
CB.SC.U4CYS24061

Akhil Charugundla
CB.SC.U4CYS24010

Bathina Manikanta Srinivas
CB.SC.U4CYS24008

Nunna Viswanadha Mukesh
CB.SC.U4CYS24044

October 21, 2026

Abstract

In Software-Defined Networking a link failure is repaired by a central controller, so recovery can wait on a controller round trip. We replicate van Adrichem et al. on Mininet and Open vSwitch and compare four recovery methods (restoration, protection with loss-of-signal, protection with BFD, and hybrid) under carrier-loss and silent link failures, on three topologies in 325 measured trials.

3.6 / 14.7 / 40.9
ms, BFD 1/5/15 ms protection
(paper: 3.4 / 15.0 / 42.1)

0 / 10
silent failures recovered
by loss-of-signal protection

21.5 → 118
ms restoration as the controller
moves 0 → 50 ms away

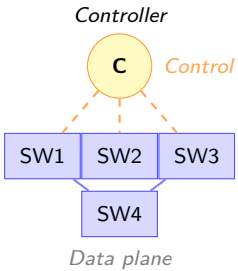
Switching to a backup takes milliseconds; detecting the failure is the real bottleneck.

Introduction — What is Software-Defined Networking?

- SDN **decouples** the control plane (routing decisions) from the data plane (packet forwarding)
- A centralised **controller** programs flow rules into switches via OpenFlow
- Gives operators a **global view** of the network — faster, more consistent policy enforcement

Why SDN?

- Traditional networks embed control logic in every switch — hard to update, error-prone
- SDN simplifies management: one controller, programmable behaviour, vendor-neutral forwarding



Introduction — The Problem and Our Objectives

Carrier-grade networks expect traffic to be recovered within **50 ms** of a failure. In SDN the switch that sees the failure usually does not know an alternative path; the controller does.

Carrier loss (fibre cut)

- Physical signal disappears; the port reports it
- Real NICs can be slow: OVS took ∼4.8 s; in Mininet (veth) it is near-instant

Silent (data-plane) failure

- Link stays up but drops every packet
- **No** carrier-loss event — needs active probing (BFD)

Objectives

1. **Replicate** van Adrichem et al. on an open Mininet / Open vSwitch testbed
2. **Compare** four recovery methods under both failure types
3. **Measure** how controller distance affects each method
4. **Package** everything so anyone can re-run it

Methodology — Four Recovery Methods

1. **Restoration** (reactive): switch reports the failure → controller recomputes a path → installs new rules. No extra state, but waits for the controller.
2. **Protection (Loss-of-Signal)**: backup pre-installed in an OpenFlow **fast-failover group**; the switch swaps locally on carrier loss. **Blind** to silent failures.
3. **Protection (BFD)**: same backup, but **BFD probes** (RFC 5880) declare the link dead. Detection lies between $(M-1)\,T_{\text{bfd}}$ and $M\,T_{\text{bfd}}$ (M=3); sees both failure types.
4. **Hybrid (BFD + re-optimisation)**: immediate BFD switch-over, then the controller moves traffic to the shortest path.

Design choice	Option A	Option B
Path setup	Reactive (restoration)	Proactive (protection)
Detection	Carrier loss (LoS)	Active probing (BFD)
Optimality	Immediate backup only	Re-optimise after failover

Methodology — Experimental Design

ID	Topology	Question	Trials
E1	Triangle	Detection: LoS vs BFD 1/5/15 ms; carrier vs silent	80
E2	5-node ring	Interior failure: crankback; adds restoration, hybrid	50
E3	USNET (24 nodes)	Scale: random cut on one of 20 long paths	120
E4	5-node ring	Controller distance 0, 5, 10, 25, 50 ms (one-way)	75
		Total	**325**

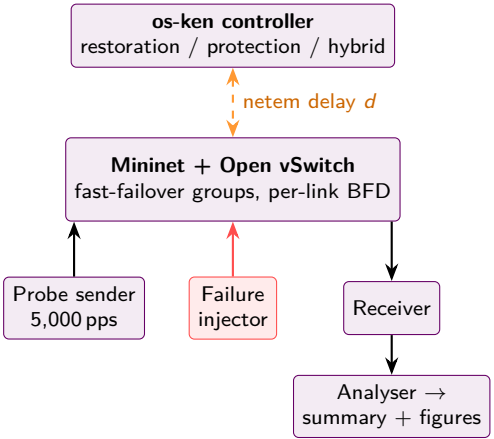
Controlled conditions

- Same failures for every method
- Controller 10 ms away (one-way) unless varied
- 10 trials per setting (30 on USNET, 5 in E4)

How recovery is measured

- Numbered UDP probe stream at 5,000 packets/s
- Recovery time = lost packets $\times$ 0.2 ms
- Same loss-based measure as the paper

Implementation — Testbed



- **Mininet 2.3** emulates switches, hosts and links
- **Open vSwitch 2.17.5**: BFD sets port liveness, which the fast-failover bucket watches
- **os-ken** controller app runs each recovery method
- **netem** adds the controller delay
- Carrier failure: interface taken down. Silent failure: all frames dropped, carrier stays up
- One **Docker image** + public GitHub repo

Bug found and fixed

OpenFlow drops a packet sent back out its own ingress port, so crankback failed. Fixed with `OFPP_IN_PORT`; every experiment re-run.

Implementation — Interactive Simulator: Carrier Loss

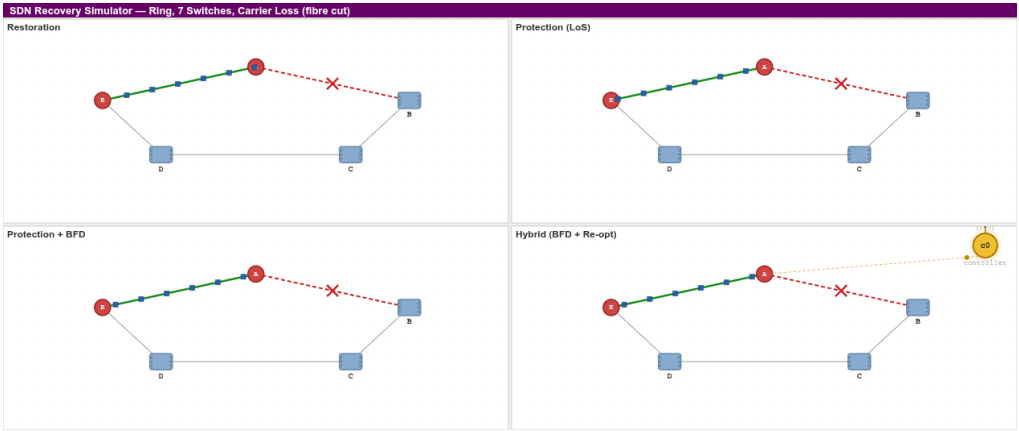


Illustration from our interactive simulator. Measured: Protection LoS is fastest at 2.1 ms (purely local switch-over).

Implementation — Interactive Simulator: Silent Failure

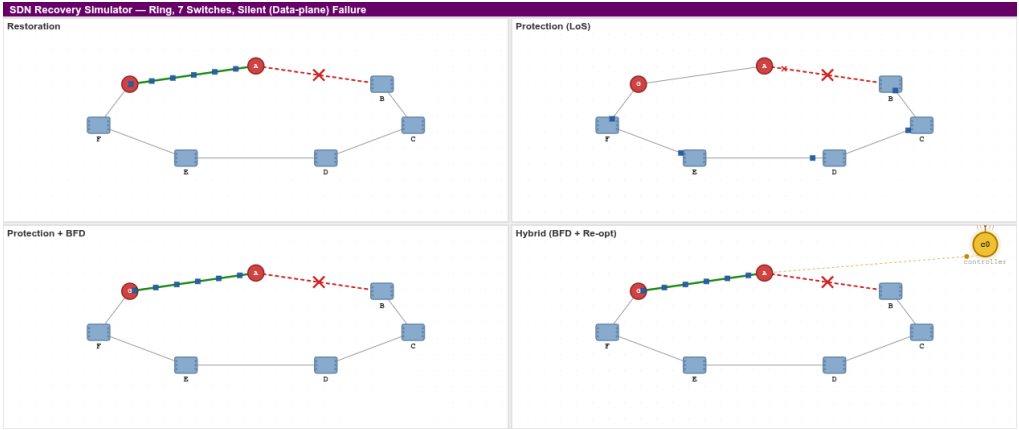


Illustration from our interactive simulator. Measured: LoS-based schemes never recover (0/10); BFD-based methods all recover.

Results — Replication of van Adrichem et al.

Protection + BFD, controller 10 ms away. All values in ms.

Protection + BFD	**1 ms**	**5 ms**	**15 ms**
Triangle — paper	3.4 ± 0.7	15.0 ± 2.2	42.1 ± 5.0
Triangle — ours (median)	3.6	14.7	40.9
Ring, crankback — paper	3.3 ± 0.8	15.4 ± 2.7	46.2 ± 5.1
Ring, crankback — ours (median)	4.8	17.7	40.7
USNET (24 nodes) — paper	–	13.6 ± 2.6	–
USNET — ours (mean)	–	16.3 ± 2.9	41.3 ± 4.9

BFD at 1 ms was fragile in software OVS (4 of 20 triangle and ring trials were anomalous; one failure was never detected), so medians are shown there. The paper reported USNET only at 5 ms; 1 ms was not run there because 86 BFD sessions at 1 ms can overload the software switch.

Results — All Methods × Both Failure Types

Measured, triangle topology (10 trials each). BFD interval = 5 ms, controller distance = 10 ms.

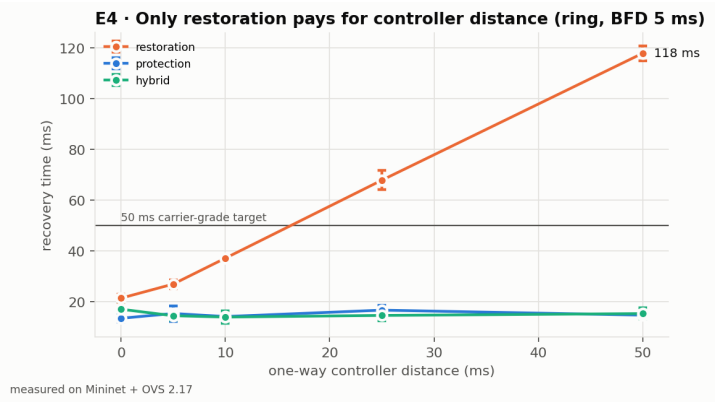
Method	Silent	Carrier
Restoration (BFD / LoS)	34.7 ms	23.8 ms
Protection (LoS)	**FAILED**	**2.1 ms**
Protection (BFD)	15.3 ms	–
Hybrid (BFD + reopt)	15.7 ms*	–

*Ring topology (the triangle has no detour to re-optimise); path re-optimised at 47.9 ms. Restoration with LoS never recovers from a silent failure. –: not measured.

Key observations

- Protection LoS is fastest for carrier loss but **cannot recover** from silent failures
- BFD protection detects both failure types in ∼15 ms
- Restoration is slowest: it waits for the controller round trip
- Hybrid gives fast failover *and* an optimal final path

Results — Effect of Controller Distance (E4)



- Restoration grows from **21.5** to **118 ms** as the controller moves $0 \rightarrow 50$ ms away
- It misses the 50 ms target once the controller is more than $\sim$16 ms away
- Protection and hybrid stay at **14–17 ms**: the switch recovers without asking the controller

Conclusion & Future Work

Conclusion

- Our testbed **reproduces** van Adrichem et al. on all three topologies
- LoS recovers in $\sim$2 ms, but only from carrier loss
- Silent failures need **BFD-based protection**
- **Hybrid**: fast failover, then an optimal path
- Only restoration pays for controller distance

Future Recovery Methods to Explore

- **In-switch (P4) detection** at hardware speed, without per-link BFD cost
- **Probe-free detection** from the traffic itself
- **ML-based recovery** that acts before a link fully fails
- **Congestion-aware** backup paths
- Recovery from **multiple, correlated** failures
- **Secure** failure signals that cannot be forged or hidden

*Takeaway: fast-failover switches in milliseconds — the **detection mechanism** is the real bottleneck.*